SGLang Runtime as a Distributed State Machine

Deep Dive into SGLang — Chapter 4

Yin Yang

Foundation Books Project

Where this chapter sits

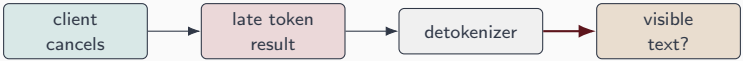
This is the chapter that turns "a scheduler that picks batches" into "a service that keeps its promises."

- Earlier chapters chose *which* work to run. This one asks: while that work moves across processes, does each request stay itself?
- SGLang is not one function call — it is several roles passing a request between them, each owning a slice of its state.
- We read those roles as a distributed state machine: a request is the *trace* of events carrying one identity, not a location.

Goal: at every handoff, name who owns the state, which message moved it, and which invariant must still hold before the client can observe a result.

The bug that motivates everything

A client cancels. The stream has gone quiet — but a token result is still in flight from the worker.



If the wrong role accepts that stale result, the service can speak *after* cancellation, or attach text to the *wrong* stream.

Everything in this chapter exists to make that handoff auditable: which role may accept the late result, and which invariant forbids it.

Roadmap

State and roles

The trace under stress

Concept to code

State and roles

A request is a trace, not a location

Definition (Request trace)
For identity r, the trace $\tau(r) = (e_0, e_1, \ldots)$ is the ordered sequence of runtime events whose payloads refer to r — arrival, input, admission, execution, partial output, terminal output, abort, pause, health, failure.

- The request moves across roles; the identity is what must survive every move.
- Correctness = can an observer still project *one* coherent $\tau(r)$ after each role boundary?

The runtime state is six owned ledgers

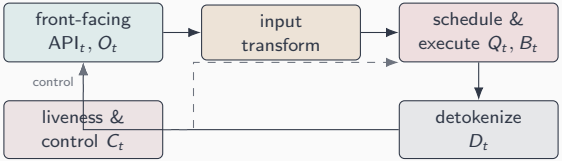
Definition (Runtime state)

$$S_t = (\mathrm{API}_t,\ Q_t,\ B_t,\ D_t,\ O_t,\ C_t).$$

API_t	API-facing obligation (owed/sealed, cancel, timing)
Q_t	waiting work
B_t	executing batch state
D_t	detokenization (held byte/token suffix)
O_t	already-emitted visible output
C_t	control-plane and liveness state

Different roles own different components. No single Python object holds S_t — that is exactly why the handoffs need checking.

Roles and the edges between them

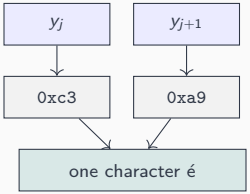


Solid edges carry the ordinary data path; dashed edges carry control events (abort, pause, health, failure). Each edge is a place request identity can be lost — so each edge is a place we audit.

The trace under stress

Streaming: tokens are not characters

The model emits token ids; the user sees *text*. They do not line up.



Streaming consistency: $c_1 \| c_2 \| \cdots \| c_k = \text{decode}(y)$, and visible chunks are append-only. D_t holds the incomplete suffix until it is safe to print.

The scheduler may change batches, not meaning

Definition (Batch semantic equivalence)
Two schedules are equivalent for $\{r_1, \ldots, r_N\}$ if every projected trace has the same terminal status and the same visible output — even when intermediate batch membership differs.

- Overlap may launch the next batch before the last result is committed.
- But request-visible commit stays in trace order — a reorder buffer for tokens.

The scheduler picks placement; the reconciliation barrier decides when a result becomes part of a trace.

Control events are transitions with an affected set

Definition (Control-plane safety)
For control event e with affected set $A(e)$: every $r \notin A(e)$ keeps its trace projection, and every $r \in A(e)$ either keeps a consistent state or moves to a terminal state — without breaking identity, streaming, or cleanup.

Abort, pause/resume, cache flush, weight update, health, introspection — all read as effects on S_t. A flush must not clear KV a live decode still reads (quiescence).

Liveness: see progress, or fail closed

- **Readiness** — roles launched, can accept work.
- **Health** — can still observe progress (or report failure).
- **Completion** — one trace reached terminal state.

Definition (Fail-closed transition)
Map an uncertain/failed internal state to one where affected traces make no silent progress, failure is observable, and diagnostics are retained: $S' = \text{failclose}(S, A)$.

A watchdog must either see the progress counter advance, or convert the stall into a diagnosable failure — never wait forever.

Concept to code

From concept to code: the overlap reconciliation barrier

The “commit in trace order” rule is the shape of the overlap loop itself.

from `python/sglang/srt/managers/scheduler.py`

```
def event_loop_overlap(self):
    self.result_queue = deque()
    while True:
        ...
        batch = self.get_next_batch_to_run()       # launch next ...
        if batch:
            batch_result = self.run_batch(batch)
            self.result_queue.append((batch.copy(), batch_result))
        if self.last_batch:                          # ... commit previous
            pop_and_process()      # process_batch_result(popleft())
        # sampling depends on the last batch (e.g. grammar): run after
        self.launch_batch_sample_if_needed(batch_result)
```

The next batch launches while the previous result waits in `result_queue`; sampling runs only after the prior result is committed. Placement overlaps; trace order does not.

The invariant checklist

Read these as a diagnostic checklist for a distributed runtime:

1. **Request identity** — every chunk on the trace its input caused.
2. **Single terminality** — at most one terminal result; nothing after.
3. **Streaming monotonicity** — visible chunks are append-only.
4. **Payload shape agreement** — index i means the same request across every array.
5. **Control safety** — unaffected traces untouched.
6. **Cleanup** — terminal r leaves the live set.
7. **Parallel agreement** — ranks agree on the logical batch.
8. **Bounded detok state & observability** — state sealed; metadata on the right trace.

What to carry forward

1. A request is a **trace of events under one identity**, owned by different roles at different times.
2. Runtime state is **six ledgers** S_t; correctness is a property of *handoffs*, not of one function.
3. Streaming, scheduling, control, and liveness each **stress the same identity contract**.
4. The runtime is correct only if **every handoff remembers who it is speaking for**.
5. These invariants are an **audit checklist** for reading the SGLang source — the spine of this chapter.

Next: continuous batching changes which requests run together — while preserving every contract we just stated.